Bounded-memory streaming redaction of personal data and credentials with a whole-document equivalence guarantee

Mounish Senisetty[1], Sandhiya Kennedy[1], Ashrith Reddy N[1], Parvathy R[1*]

[1*]Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, India.

*Corresponding author(s). E-mail(s): parvathyr@am.amrita.edu;
Contributing authors: am.ai.u4aid23058@am.students.amrita.edu;
am.ai.u4aid23056@am.students.amrita.edu;
am.ai.u4aid23014@am.students.amrita.edu;

Abstract

Logs and data exports carry personal data and credentials and often exceed memory. Chunked processing adds a silent failure: a value straddling a chunk boundary is split, missed and written out in the clear. SafeStream-Redactor treats chunking as a correctness problem. Each detector declares its longest span and the context it reads. We show that an overlap of the longest span can still leak, and prove that if the overlap covers every detector's span plus its right context, the streamed output equals a whole-document pass, parallel output equals sequential output or falls back to it, and the working buffer is independent of input size; the engine enforces this overlap. Naive 1 KiB chunking leaked 237 RedactionBench values a whole-document pass redacts; overlapping windows still missed some, SafeStream none. The condition carries over to other tools: wrapped around Presidio's recognisers, the engine reproduced whole-document output exactly once it had exposed, and we had removed, one Presidio pattern whose output depends on arbitrarily distant text. Streaming added 8.4–8.54 MiB from 8 MiB to 2 GiB. On 14,626 real credentials labelled by people, the pattern tiers fully redacted 0.55 against 0.40 for gitleaks, robust to resampling by repository, at a higher false-alarm rate. On 1,014 held-out court judgments, a free-text configuration frozen before scoring reached character F1 0.67 against 0.54 for Presidio (0.76 against 0.63 on Wikipedia biographies). Yet no configuration fully protected more than 17.9% of RedactionBench documents.

Keywords: Data redaction, Secret detection, Data loss prevention, Streaming algorithms, Bounded memory, Personally identifiable information

1 Introduction

Text artefacts that leave an organisation, such as an application log attached to a support ticket, a database export handed to an analyst, or a packet capture shared with a vendor, routinely carry two classes of sensitive value: personal data (e-mail addresses, telephone and card numbers, national identifiers, names) and credentials (application programming interface keys, access tokens, private keys, passwords). Data-loss-prevention systems inspect such content before it crosses a boundary [1], and measurement studies show that credentials leak into shared artefacts at scale [2, 3].

Two families of tools address the problem. Personal-data engines such as Microsoft Presidio [4] combine pattern recognisers with a statistical named-entity model but ship no credential recognisers. Secret scanners such as gitleaks [5] and TruffleHog [6] target credentials in source repositories, and their accuracy varies widely across secret types [7]. The artefacts that most need redaction are often too large to analyse in one call, so they are processed in pieces. A chunked redactor can then fail silently: a value that straddles a chunk boundary is presented to the detector in two halves, neither of which matches, and both halves are written out in the clear. The same problem is long established in network security, where signatures split across packets motivated stream reassembly in intrusion detection [8] and streaming modes in regular-expression engines [9, 10]. In redaction, however, detectors read context around a match ("Patient ID:" before a number, a key name before a token), confidences are adjusted by neighbouring text, and overlapping detections are merged before the text is rewritten. Overlapping windows are the obvious remedy, but how much overlap is enough depends on all of these, and the obvious answer, an overlap as long as the longest match, is wrong: a match whose right context is cut off at the window edge can be rejected there, so the boundary does not retreat out of it and its first part is emitted in the clear. None of the redaction or secret-scanning systems we examined states when its chunked output equals its whole-document output.

This paper answers that question for a concrete system, SafeStream-Redactor (hereafter SafeStream). Its contributions are the following.

1. **A tight, per-detector overlap condition for context-dependent redaction.** Every detector d declares an upper bound L_d on the span it can emit and bounds $C_{\mathrm{left},d}$ and $C_{\mathrm{right},d}$ on the context it reads (Assumptions A1 and A2). We prove that an overlap $w \geq \max_d \max(L_d + C_{\mathrm{right},d},\, C_{\mathrm{left},d})$ makes the streamed output equal to the whole-document output for every chunking, and show that the term $L_d + C_{\mathrm{right},d}$ cannot be dropped: an overlap of $\max_d L_d$, the natural choice, can leak (Section 5). We also measure the failure the condition prevents: on the test corpora, naive chunking leaked up to 237 gold values that a whole-document pass redacts, and overlapping windows still leaked some, while SafeStream's output was identical to the whole-document output under every chunking tried (Section 7.1).

The underlying locality argument is standard; what is new is the condition for pipelines whose detectors read context on both sides, adjust confidences and resolve overlaps jointly, which pattern-matching engines that carry automaton state across blocks do not address. The condition is not tied to our detectors: wrapped around Presidio's recognisers, the engine made their chunked output identical to their whole-document output, and the same check exposed a Presidio pattern whose output depends on arbitrarily distant text (Section 7.1).

2. **A guarantee enforced by the code, not chosen by hand.** The engine computes the condition from the detectors' declarations and refuses to run with a smaller overlap; regression and differential tests check that the declarations hold. For the default pipeline the requirement is exactly the default overlap of 8192 characters.
3. **Parallel redaction that cannot lose or duplicate text.** A seam protocol between worker processes makes the parallel output equal to the sequential output under the same condition, and falls back to sequential processing on any disagreement, so no character is lost or emitted twice whatever the detectors do (Appendix A).
4. **Bounded memory, measured.** The working buffer is bounded independently of input size, and peak memory stays flat across a 256-fold range of input sizes, where whole-document redaction needs about 28 times the input size (Section 7.2).
5. **A pre-registered detection evaluation on real data.** Equivalence says nothing about how good the detectors are. Beyond held-out test splits, we evaluate on four real, human-labelled corpora (system logs, 14,626 credentials from 337 repositories, court judgments and Wikipedia biographies), each with its protocol and manifest committed before any system was run, against Presidio and two secret scanners, with paired bootstrap intervals at the document, cluster and two-stage levels (Sections 6 and 7). SafeStream fully redacts more real credentials than gitleaks, robustly but with more false alarms; a free-text configuration, designed on development data and confirmed on 1,014 held-out judgments and on a second genre, detects more personal data than Presidio; and we report the margins that do not survive, including a reversal on court judgments for the configuration originally evaluated (Sections 7.8 and 7.9).

Two scope limits are stated at the outset. First, the guarantee concerns equivalence rather than accuracy: streaming does not improve detection, it makes the streamed result the same as the whole-document result, so every detection error of the whole-document pipeline is also an error of the streamed one. Second, the evaluation has known weaknesses that we quantify rather than omit: one corpus, RedactionBench, was inspected during rule development (Section 7.5); two corpora are synthetic, and on real court judgments the evaluated named-entity configuration detected less personal data than Presidio until a free-text configuration was added (Section 7.8); and fewer than one RedactionBench document in five was fully protected by any configuration we evaluated (Section 7.5). Section 8 states when the approach is useful despite these limits and what evidence a deployment would need.

2 Background and related work

Personal-data detection and de-identification

Rule-based de-identification of clinical text dates back to lookup tables and regular expressions applied to nursing notes [11]; Philter refined the approach on a large annotated clinical corpus [12]. The 2014 i2b2/UTHealth shared task established annotated corpora and evaluation practice [13], and recurrent neural networks reached strong results on them [14]. Presidio [4] combines pattern recognisers with spaCy or transformer models and ships an evaluation toolkit with a synthetic dataset [15]. Lison et al. argue that sequence labelling alone does not amount to anonymisation and call for explicit disclosure-risk measures [16]; the Text Anonymization Benchmark operationalises this for court judgments [17]. Named-entity recognition underpins the free-text part of these systems, from the CoNLL-2003 shared task [18] to transformer encoders [19, 20] and generalist span models [21]. Pseudonymisation with a keyed hash such as HMAC [22] preserves referential integrity but reveals equality between values, so it is not anonymisation in the sense of k-anonymity [23]. Redaction as studied here is narrower than anonymisation: it removes spans that detectors recognise and makes no claim about re-identification from what remains.

Credential and secret detection

Meli et al. measured secret leakage across public GitHub repositories [2]. Approaches range from vendor-specific regular expressions [24] to machine-learned filters that reduce false positives [25] and neural detectors for free-form passwords [26]. SecretBench provides a labelled corpus of candidate secrets [27], and a comparison of nine tools attributed false positives to generic patterns and entropy heuristics and false negatives to missing rules [7]. Krause et al. studied how developers handle leaked secrets [3]. Shannon entropy [28] is the usual signal for undocumented high-entropy secrets. The Nosey Parker rule set [29] supplies the labelled examples we evaluate on.

Detection across segment boundaries

That a pattern split across units of input can evade a detector is well known in network security: Ptacek and Newsham showed that intrusion detectors which do not reconstruct streams as the end host does can be evaded [8], and Hyperscan [9] provides a streaming mode whose matches do not depend on how the data is split into blocks [10]. These systems carry automaton state across boundaries, which makes pattern matching exact but says nothing about what happens after a match: validators, context-dependent confidences and joint overlap resolution all read text the automaton has already consumed. A redaction pipeline differs in three ways that affect correctness: detectors consult bounded context on both sides of a match, confidences depend on nearby text, and overlapping detections are resolved jointly before the output is rewritten. Overlapping windows are a common way to process long inputs, and the stream model [30] formalises computation in bounded memory; linear-time regular-expression matching goes back to Thompson [31]. Among the redaction, de-identification and secret-scanning systems cited above, we found none that states when chunked output equals whole-document output or how the overlap must be sized. We

do not claim that the underlying locality argument is new; the contribution is its instantiation and enforcement for context-dependent redaction.

3 Problem statement and threat model

Definition 1 (Detections and redaction) A text T is a finite character string, and $T[a{:}b]$ is its half-open substring. A *detection* is a tuple (i, j, τ, c) with $0 \le i < j \le |T|$: a half-open span, an entity type and a confidence in $[0, 1]$. Each detection is produced by one detector d of a finite set $\mathcal{D}$. A *pipeline* Π maps T to a set of pairwise-disjoint detections. *Redaction* rewrites T by replacing each span in $\Pi(T)$ according to a policy (replace, mask, hash, keyed pseudonym or encryption). The replacement of a span depends only on its type and its text, so redaction is a deterministic function of T and $\Pi(T)$ that acts on each span separately.

Definition 2 (Leak) Given a set G of gold sensitive spans for T, a *leak* is any character of a span in G that is not covered by a span in $\Pi(T)$. A gold span with at least one uncovered character is *leaked*, because a partly visible card number or key can often be reconstructed or narrowed.

Adversary and setting

The adversary is a later reader of the redacted output, for example the recipient of a shared log. The input may be larger than memory and arrive as an arbitrary sequence of chunks. The system runs locally and makes no network calls, so the unredacted text does not leave the host.

What is in scope

(i) That chunked and parallel processing introduce no leak beyond those of the whole-document pipeline; (ii) bounded working memory; (iii) the detection quality of the whole-document pipeline, measured empirically.

What is out of scope

Detection completeness: the pipeline misses values its detectors do not recognise, and the guarantee does not change that. Adversarial evasion: an author who encodes, splits or obfuscates a secret on purpose is not modelled. Inference from residual text: quasi-identifiers and context that remain after redaction may still allow re-identification [16, 23]. Side channels such as output length. Bounded memory is a resource property, not a security property.

4 Design

Figure 1 shows the architecture. The reference pipeline Π runs every configured detector on a text, adjusts confidences with a bounded contextual scorer, drops detections below a threshold (0.5 by default) and resolves overlaps into a disjoint set.

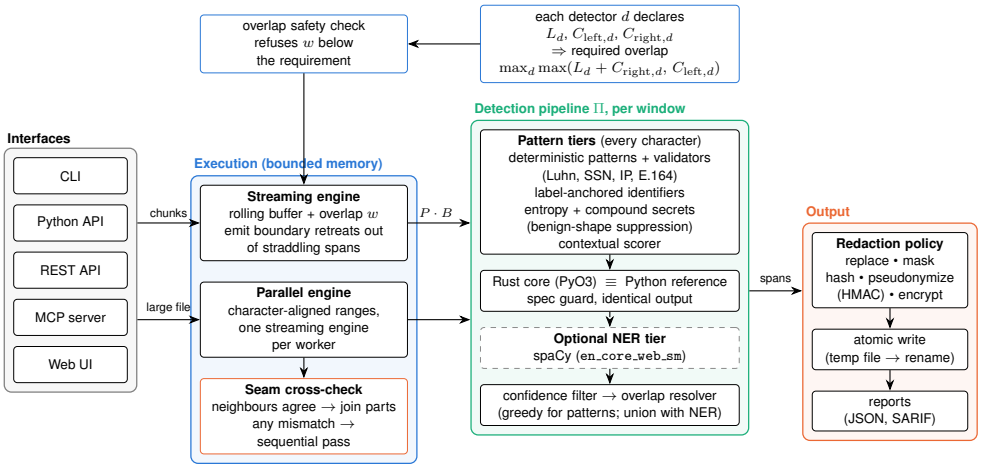


Fig. 1 System architecture. Interfaces feed text to the streaming engine or, for a large file, to the parallel engine, whose workers each run a streaming engine and whose seams are cross-checked. Each window passes through the detection pipeline Π: the pattern tiers run on every character, on the Rust core or the identical Python reference. The dashed named-entity tier (spaCy) is optional. The resolver produces disjoint spans, which the redaction policy rewrites and an atomic write commits. The overlap w is checked against the span and context bounds the active detectors declare.

4.1 Detection tiers

Deterministic patterns with validators

Credential formats with a fixed shape (cloud access keys, provider tokens, JSON Web Tokens, PEM private-key blocks) and structured personal data (e-mail, telephone, card and social-security numbers, IPv4 and IPv6 addresses) are matched by regular expressions. A match becomes a detection only if it passes a validator: the Luhn checksum for card numbers [32], area, group and serial rules for social-security numbers, octet ranges for IPv4, address parsing for IPv6, and an E.164 shape check for telephone numbers. A generic key-assignment rule matches values that follow keys such as `api_key`, `token` or `password`.

Label-anchored identifiers

Unchecked numeric identifiers such as medical record, account or licence numbers are ambiguous in isolation, so this tier promotes a value to a detection only when a cue word that names it (for example `Patient ID:`) appears within 96 characters to its left.

Entropy tier with benign-shape suppression

Unstructured candidate runs of 20 to 8192 characters over a key alphabet are scored by length, character-class mix and Shannon entropy [28], with a lower threshold for hexadecimal runs; a compound variant covers runs with embedded symbols. Entropy alone cannot separate a secret from other random-looking strings. During this study

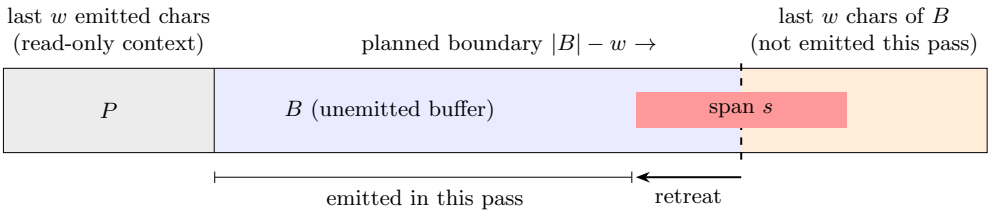


Fig. 2 One pass of Algorithm 1. The detector reads $P \cdot B$; only text before the planned boundary may be emitted, and the boundary retreats out of any span that crosses it, so s is carried to the next pass and detected whole.

every generated UUID, commit hash, SHA-256 digest and base64 image payload in the secrets development split was flagged (Section 7.4). The tier therefore suppresses three shapes: a canonical UUID, a 40- or 64-character hexadecimal value preceded by a digest cue (`commit`, `sha256`, `hash` and similar), and the payload of a `data:` URI. Suppression is vetoed when a secret-like key (`key`, `token`, `secret`, `password` and similar) names the value, so `SAUCE_ACCESS_KEY=`*uuid* remains a secret. The check reads at most 32 characters of left context.

Contextual scorer

A 48-character left window raises or lowers confidences when trigger words are present and emits a small set of trigger-gated detections of its own.

Optional named-entity tier

For names, organisations and locations, an optional tier runs a spaCy pipeline (`en_core_web_sm` in our experiments).

4.2 Overlap resolution

Pattern detections that overlap are resolved best-first by confidence, then by length, so a card number beats a telephone number embedded in it. Named-entity detections are handled differently: any cluster of overlapping spans that contains a named-entity span becomes one span from the cluster's leftmost start to its rightmost end, labelled by its best pattern member if one exists. Without this union rule, a PERSON tag on `john.doe` could replace the e-mail detection for `john.doe@example.com` and leave the domain exposed.

4.3 Streaming engine

Algorithm 1, a simplified form of `engine.stream_windows` illustrated in Figure 2, keeps a buffer of unemitted text and the last w characters already emitted. Each pass detects over their concatenation, so detectors see real left context rather than an artificial edge, and emits only the part of the buffer that lies at least w characters before its end. If a detection crosses the planned emit boundary, the boundary retreats to the detection's start and the detection is carried into the next pass. Small chunks

Algorithm 1 Streaming detection with overlap w

Require: chunks $c_1, c_2, \ldots$; pipeline Π; overlap w; minimum emit size $m \geq w$

$P \leftarrow \varepsilon$ ▷ emitted text kept as read-only left context
$B \leftarrow \varepsilon$ ▷ buffer of unemitted text
for each chunk c **do**
  $B \leftarrow B \cdot c$
  if $|B| < w + m$ **then**
    continue
  end if
  $S \leftarrow \{(i - |P|,\ j - |P|,\ \tau,\ \kappa) : (i, j, \tau, \kappa) \in \Pi(P \cdot B),\ i \geq |P|\}$
  $e \leftarrow |B| - w$ ▷ planned emit boundary
  while some $(i, j, \tau, \kappa) \in S$ has $i < e < j$ **do**
    $e \leftarrow i$ ▷ retreat out of the straddler
  end while
  if $e \leq 0$ **then**
    continue ▷ a span pins the buffer; read more
  end if
  emit $B[0{:}e]$ with $\{(i, j, \tau, \kappa) \in S : j \leq e\}$
  $P \leftarrow$ last w characters of $P \cdot B[0{:}e]$
  $B \leftarrow B[e{:}]$
end for
emit B with $\{(i - |P|,\ j - |P|,\ \tau,\ \kappa) : (i, j, \tau, \kappa) \in \Pi(P \cdot B),\ i \geq |P|\}$

are coalesced until at least m characters can be emitted, which bounds the cost of rescanning the overlap per emitted character regardless of how the input is chunked. The defaults are $w = 8192$ characters and $m = 65{,}536$ characters.

4.4 Parallel engine

For large files the parallel engine splits the input into character-aligned byte ranges of at least $4w$ characters, one per worker process. Each worker reads w characters of scanned padding and a further w characters of context-only prefix around its interior boundaries (seams). A span that crosses a seam belongs to the left worker, and the right worker begins emitting at that span's end. Neighbouring workers compute the seam position independently, and the coordinator compares their reports; on any disagreement it discards the parallel result and redacts the file sequentially.

4.5 Implementation

The pure-Python implementation is the reference. An optional Rust core implements the pattern tiers. At start-up it checks that its compiled pattern sources equal the Python ones, and it hands a window back to Python in the rare cases where Unicode case folding or floating-point summation could make the two disagree.

5 Formal analysis

Let $S_w(T)$ be the set of detections emitted by Algorithm 1 for some chunking of T, mapped back to absolute offsets, and $P_{w,k}(T)$ the output of the parallel engine with k workers. Each detector $d \in \mathcal{D}$ declares three constants: the longest span it emits, L_d, and the context it reads to the left and right of a span, $C_{\text{left},d}$ and $C_{\text{right},d}$. The two assumptions are stated per detector and must hold for every input text, not only for T, because the engine applies Π to windows.

Assumption A1 (Bounded span) For every text X, every detection of detector d in $\Pi(X)$ has length $j - i \le L_d$.

Assumption A2 (Bounded context) Let X and Y be texts and δ an offset with $X[a{:}b] = Y[a{+}\delta : b{+}\delta]$. Let (i, j) be a span of detector d such that either $i - C_{\text{left},d} \ge a$ or $a = a{+}\delta = 0$, and either $j + C_{\text{right},d} \le b$ or both $b = |X|$ and $b + \delta = |Y|$. Then $(i, j, \tau, \kappa) \in \Pi(X)$ if and only if $(i + \delta, j + \delta, \tau, \kappa) \in \Pi(Y)$.

In words, A2 says that whether a span of d is in the final detection set, and with which type and confidence, depends only on the $C_{\text{left},d}$ characters before it, the span itself and the $C_{\text{right},d}$ characters after it, where a window edge that coincides with an edge of both texts counts as real. The pipeline requirement is

$$w^\star \;=\; \max_{d \in \mathcal{D}} \; \max\big(L_d + C_{\text{right},d},\; C_{\text{left},d}\big). \tag{1}$$

Theorem 1 (Streaming equivalence) *Under A1 and A2, if $w \ge w^\star$ and $m \ge w$, then $S_w(T) = \Pi(T)$ for every text T and every chunking of T.*

Corollary 1 (Pipeline-wide sufficient condition) *Let $L_{\max} = \max_d L_d$, $C_{\text{left}} = \max_d C_{\text{left},d}$ and $C_{\text{right}} = \max_d C_{\text{right},d}$. Then $w \ge \max(L_{\max} + C_{\text{right}}, C_{\text{left}})$ implies $w \ge w^\star$, so Theorem 1 applies.*

The corollary is the intuition: the overlap must hold the longest span together with the widest right context, and the widest left context. It is sufficient but not necessary, because it pairs the longest span with a right context that may belong to a different detector. Equation (1) is the condition the engine checks (Section 5.1).

Why the right context is added to the span

An overlap of $\max_d \max(L_d, C_{\text{left},d}, C_{\text{right},d})$, which bounds each quantity separately, is not enough. Take a detector d that rejects a candidate whose right context is missing, and an overlap $w < L_d + C_{\text{right},d} - 1$. A chunking can end a pass so that a span $s = (i, j)$ of $\Pi(T)$ starts one character before the planned boundary $e_0 = |B| - w$ and has length L_d. Its right context then extends $(L_d - 1) + C_{\text{right},d} > w$ characters beyond e_0, past the end of the buffer. The window therefore rejects s. If no other detection straddles e_0, the boundary does not retreat, and the pass emits $T[i : e_0]$,

the first character of s, unredacted. A shorter span ending before e_0 can be emitted in its place. The bound in Eq. (1) is thus tight up to one character. It is also why the safe-region step in the proof below adds $C_{\text{right},d}$ to L_d.

Proof of Theorem 1 Fix a non-final pass. Let p be the absolute position of $B[0]$, $q = p + |B|$ and $e_0 = q - w$ the planned boundary. The window is $W = T[p - |P| : q]$, where $|P| = \min(w, p)$ because P holds the last w emitted characters.

Safe region. Let $s = (i, j, \tau, \kappa)$ be a span of detector d, in absolute offsets, with $p \le i < e_0$ that belongs to $\Pi(W)$ or to $\Pi(T)$. By A1, $j \le i + L_d < q - w + L_d \le q - C_{\text{right},d}$, using $w \ge L_d + C_{\text{right},d}$. On the left, $i - C_{\text{left},d} \ge p - w$ because $w \ge C_{\text{left},d}$; if $p < w$, the window starts at position 0 of T. A2, applied to $X = W$ and $Y = T$, then gives $s \in \Pi(W) \Leftrightarrow s \in \Pi(T)$. So the spans of the window that start in $[p, e_0)$ are exactly the spans of $\Pi(T)$ that start there.

Retreat. $\Pi(W)$ is pairwise disjoint, so at most one span s^* straddles e_0, and it starts before e_0, so it belongs to $\Pi(T)$. The boundary becomes $e = i^*$ if s^* exists and $e = e_0$ otherwise. In the first case, $i^* > e_0 - L_d \ge p + m - L_d \ge p$, because $|B| \ge w + m$ and $m \ge w \ge L_d$. So $e > p$, and the pass emits at least one character; the pinned branch is never taken. No span of $\Pi(T)$ straddles e. If $e = e_0$, such a span would start before e_0 and so be the straddler of W. If $e = i^*$, it would overlap s^*, which disjointness rules out.

Emission. The pass emits $T[p{:}e]$ with the window spans contained in it. By the safe region these are exactly the spans of $\Pi(T)$ contained in $[p, e)$. The final pass has $q = |T|$, so the right-hand clause of A2 holds for every span, and it emits exactly the spans of $\Pi(T)$ inside $[p, |T|)$. The emitted segments partition T, and no span of $\Pi(T)$ crosses a segment boundary. Every span of $\Pi(T)$ is therefore emitted exactly once, with the type and confidence it has in $\Pi(T)$, and nothing else is emitted. □

Theorem 2 (Parallel identity with fail-safe fallback) *(i) Under the conditions of Theorem 1, and with ranges of at least $4w$ characters, every seam report agrees and $P_{w,k}(T)$ equals the sequential output byte for byte. (ii) Without any assumption on the detectors, the output is either assembled from parts whose seam reports all agree, so that every character of T is emitted exactly once, or it is the sequential output.*

Theorem 3 (Bounded working buffer) *Let c be the length of the longest input chunk. Under the conditions of Theorem 1, the unemitted buffer of S_w never exceeds $w + m + c$ characters, and each detection window never exceeds $2w + m + c$, independent of $|T|$. Each parallel worker additionally holds back at most $2w$ characters plus one segment, so the parallel engine needs $O\big(k\,(w + m + c)\big)$ characters.*

Because redaction acts on each span separately (Definition 1), Theorem 1 implies byte-identical redacted output. The proofs of Theorems 2 and 3 are in Appendix A, and Online Resource 1 maps each step to the implementation. For `redact_file`, $c = m = 65{,}536$ and $w = 8192$, so the buffer stays below 139,264 characters and each window below 147,456 characters. The proofs were not machine-checked.

5.1 What is derived and what is assumed

Each detector declares $(L_d, C_{\text{left},d}, C_{\text{right},d})$. The engine computes $w^\star$ from Eq. (1) over the active detectors and refuses to start with a smaller overlap unless the caller

Table 1 Declared streaming bounds of the evaluated detectors, in characters, at the evaluated commit. The requirement of a detector is $\max(L_d + C_{\text{right},d},\, C_{\text{left},d})$; the pipeline requirement $w^\star$ is the largest requirement among active detectors.

Detector d	L_d	$C_{\text{left},d}$	$C_{\text{right},d}$	Requirement
Deterministic patterns	8192	0	0	8192
Label-anchored identifiers	64	96	8	96
Entropy tier	8192	32	0	8192
Compound entropy tier	8192	0	0	8192
Contextual scorer	9	48	0	48
spaCy named-entity tier (optional)	256	256	256	512
Default pipeline, $w^\star$				8192
Default pipeline with spaCy, $w^\star$				8192

explicitly overrides it. Table 1 lists the declarations. The detectors that emit the longest spans (private-key blocks and entropy candidates, $L_d = 8192$) read no right context. The only detector with right context among the default tiers, the label-anchored tier, emits spans of at most 64 characters. Hence $w^\star = 8192$, and the default overlap $w = 8192$ satisfies Theorem 1 exactly, with or without the spaCy tier. The pipeline-wide Corollary 1 would ask for $8192 + 8 = 8200$ characters; that is why the per-detector form is the one the engine enforces. A regression test fails if any declaration changes, and a differential test probes each detector to check that its real context dependency does not exceed its declaration.

Two parts of A1 and A2 are *not* derived from declarations. They should be read as assumptions corroborated by tests rather than proven properties of the code. (i) The confidence adjustment and threshold must not reach beyond the declared context of the detector whose span they adjust. The contextual scorer's 48-character left window is within w for every detector, but it is not added to the other detectors' declarations. (ii) Overlap resolution must not propagate decisions along chains of overlapping candidates beyond the declared contexts, and the named-entity union rule must not create spans longer than the declared bounds. For the pattern tiers, whose spans are bounded and whose overlaps are short, the equivalence tests in Section 7.1 found no counterexample. A statistical model's output can depend on its whole input window, so for the spaCy tier the declared bounds of 256 characters are declarations only, and the theorems should not be read as covering it. Empirically, its streamed output matched the whole-document output in every chunking we tried (Section 7.1).

Failure modes

Lowering the overlap below $w^\star$ is refused by default. If a caller overrides that, the parallel engine still cannot lose or duplicate characters (Theorem 2(ii)), but the sequential engine can split a long span. A detector that understates its context, or one that needs document-wide context, is outside the guarantee.

6 Experimental methodology

The evaluation addresses seven questions. **RQ1**: do the implementation's streamed, parallel and native outputs match the reference in practice, and how often does chunking without the guarantee leak? **RQ2**: how do memory and throughput scale? **RQ3**:

how well do the pattern tiers detect credentials compared with purpose-built scanners? **RQ4**: how well does SafeStream detect personal data compared with Presidio, with and without a named-entity model? **RQ5**: how much protection does it give at the level of a whole document, and how much of that depends on rules written after seeing the benchmark? **RQ6**: how well do the systems detect personal data in real system logs? **RQ7**: how well do they detect personal data in real free text annotated by people other than the authors?

6.1 Corpora and splits

Table 2 lists the test splits. Each corpus was split once into development and test halves with a fixed seed. The manifests list document identifiers and the SHA-256 of the source data, and the loader refuses a source that no longer matches. Development, error analysis and tuning used development halves only, with the exception of RedactionBench described below; test halves were scored after the code was frozen, with one disclosed exception (Section 7.4).

- **presidio-research** [15] (synthetic): Presidio's template-generated evaluation set; 750 test documents from 200 templates.
- **secrets** (semi-synthetic): Nosey Parker's labelled rule examples [29] plus 200 generated non-secrets; split by rule, so the test half holds unseen vendor formats.
- **RedactionBench** [33]: 200 documents in eleven domains, mostly seeded from real sources, with *mandatory* and *contextual* spans; inspected during rule writing, so not held out (Section 7.5).
- **OpenSSH** and **Linux logs** (real): Loghub's 2,000-line samples [34], one document per line, with gold from labels on Loghub's log templates (remote IP addresses and account names), verified and frozen before scoring; the Linux labels were drafted only after the user-name tier of Section 7.7 was committed, so that corpus is held out from it.
- **TAB** (real, human gold): 127 English judgments of the European Court of Human Rights annotated by one to ten people each for direct and quasi identifiers [17]; its development split (127) designed and its training split (1,014, held out) confirmed the free-text configuration of Section 7.9.
- **Wikipedia biographies** (real, human gold, held out): 553 summaries annotated with TAB's scheme [35], a second genre for that configuration.
- **CredData** (real credentials, human gold, held out): lines of real files from 337 repositories labelled by people as credential or not [36], values obfuscated by the dataset's builder.

Online Resource 1 (Section S18) gives each corpus's revision, labelling rule, counts and the commits at which its manifest and protocol were frozen; every real-data corpus was frozen before any system was run on it.

6.2 Systems and configurations

SafeStream was run with the Rust core and default tiers (*pattern*), with the opt-in date tier (*+dates*), and with the spaCy `en_core_web_sm` tier (*+spaCy*). **Presidio**

Table 2 Test splits. Gold-free documents contain no gold span; any detection on them is a false alarm. Clusters are the resampling units of the cluster and two-stage bootstrap (Section 6.4).

Corpus	Documents	Gold spans	Gold-free	Bytes	Clusters
presidio-research	750	1,394	63	63,678	200 templates
secrets	313	293	97	46,595	96 rules + 5 templates
RedactionBench	100	25,592	0	1,378,267	11 domains
OpenSSH logs (real)	2,000	2,491	148	221,218	27 templates
Linux logs (real)	2,000	1,876	407	212,487	118 templates
TAB judgments (real)	127	6,201[1]	0	652,935	documents
TAB held-out (real)	1,014	59,143[1]	0	8,328,627	documents
Wikipedia bios (real)	553	7,527[1]	0	270,206	documents
CredData (real)	63,706	14,626	49,517	17,345,610	337 repositories

[1]Union over annotators, overlapping mentions merged; TAB is scored against each annotator's gold separately (Section 6.3).

2.2.355 [4] was run as its pattern recognisers alone (*regex*) and as the full analyser with spaCy `en_core_web_sm` (*sm*) and `en_core_web_lg` (*lg*), with spaCy 3.8.13 and models 3.8.0. **gitleaks** 8.28.0 [5] and **TruffleHog** 3.90.8 [6] were run with default rules and default false-positive filtering; TruffleHog ran with `--no-verification`, so no example credential was sent to a live service. A sensitivity run repeated both scanners with their false-positive filters disabled (Section 7.3). Each scanner ran once over a directory with one file per document, and findings were mapped back to character offsets; TruffleHog's composite findings (for example `user:token`) were credited for each component that appears in the text. Presidio received documents longer than 20,000 characters in line-aligned pieces of at most that length; no gold span in any test split crosses a piece boundary, so the splitting cost Presidio no recall. All SafeStream thresholds are defaults; no threshold was tuned on test data. The RedactionBench dev-selected configurations (Section 7.5) were chosen on the development half by a rule fixed in advance.

6.3 Metrics

Let G and R be the sets of gold and redacted character positions in a document. Character precision and recall are $|G \cap R|/|R|$ and $|G \cap R|/|G|$, summed over documents, and F1 is their harmonic mean. All metrics are type-agnostic: a value counts as protected whatever label it receives. The *fully redacted rate* is the share of gold spans with every character redacted. The *false-alarm document rate* is the share of gold-free documents with at least one detection. On RedactionBench, *mandatory recall* is character recall over mandatory spans, and the *no-leak document rate* is the share of documents in which every mandatory character is redacted (Section 7.5 discusses how it relates to recall). TAB is scored as its authors prescribe [17]: recall against each annotator's gold, pooled over document–annotator pairs; precision counting a redacted character as correct if any annotator marked it; and *entity recall* for direct and quasi identifiers, under which an entity counts as protected only if every one of its mentions is fully redacted. TAB's information-weighted precision needs a language model and is not computed.

6.4 Statistical analysis

We report 95% percentile bootstrap intervals with 2,000 resamples and seed 0 [37]. All systems evaluated on a corpus are scored on one shared resample matrix, so every difference between two systems is paired; we report the median paired difference and call it significant only when its paired interval excludes zero. Documents are not independent. Secrets positives cluster by rule, presidio-research documents share generator templates, and RedactionBench documents share domains. We therefore use three resampling schemes on the same per-document counts [38]: *document* (documents drawn independently), *cluster* (whole clusters drawn with replacement, all their documents kept) and *two-stage* (clusters drawn with replacement, then documents drawn with replacement inside each drawn cluster). Table 2 lists the clusters. Tables 4 and 6 give document-level intervals. Table S9 of Online Resource 1 repeats the planned comparisons under all three schemes, and we call a difference *robust* when it is significant under all three. RedactionBench has only eleven clusters, so its cluster intervals are themselves imprecise; we read them as a sensitivity analysis rather than as an exact test. No correction for multiple comparisons was applied; the comparisons in the abstract were planned, the ablation comparisons were not. The analysis was not pre-registered.

6.5 Environment

Accuracy runs used a 4-vCPU Intel Xeon virtual machine at 2.80 GHz with 15 GB RAM, Linux and CPython 3.13. Throughput figures come from separate runs, five per configuration, on the same machine with no other benchmark running; the throughput recorded during accuracy runs is not used. Memory was measured as the peak resident set size of a fresh process per point minus its resident set size after imports, so native allocations are included; it is reported in MiB. Re-running every test-split system for the cluster analysis reproduced each point estimate and document-level interval to three decimals. The held-out results come from commit `0765dae`, the results after the entropy fix (Section 7.4) from `3d2c6ac`, and the pre-exposure RedactionBench results from `874e105`. On the real-data corpora the evaluated configurations ran at `3d2c6ac` with the pure-Python backend, whose output equals the Rust core's; the user-name tier and the current default profile ran at the current code, and the free-text configuration at `9c10975`, where it was frozen. All commits are on the main branch of the public repository.

7 Results

7.1 RQ1: equivalence, backend parity and leaks caused by chunking

On every test split the Rust core and the pure-Python reference produced identical predictions (presidio-research 178 spans, secrets 298, RedactionBench 3,167, from the frozen commit). The repository's equivalence suite asserts byte-identical streamed, whole-document and parallel output over exhaustive small chunkings, property-based generated inputs, seam-straddling entities and forced fallbacks, and also checks native

parity and declared context; it passed 133 of 133 tests at the final commit. These tests corroborate the theorems for the configurations they exercise; they are not proofs, and they do not exercise the named-entity tier's full input dependence.

How often chunking alone leaks

The theorems say that chunking cannot add leaks; they do not say how many leaks chunking adds without them. We concatenated each test split into one stream and ran the same pattern pipeline four ways (Table 3; `benchmarks/chunking_leak.py`): *whole*, the reference $\Pi(T)$; *naive*, fixed chunks of c characters; *overlapping windows* of $c + o$ characters advancing by c, keeping the detections that start in their own c characters ($o = 256$ and $o = 8192$, the longest declared span); and *SafeStream*, Algorithm 1 at its default overlap and $m = w$, over five random chunkings with chunk lengths uniform in $1..2c$. We also ran a dense 4.2 MB generated log (56,153 gold values) and Loghub's real logs of 16 systems [34] (4.4 MB, no redaction labels, so only missed reference detections count).

Naive chunking leaked gold values that the whole-document pass redacts on every corpus: at 1 KiB, 237 RedactionBench values, 7 on presidio-research, 5 on secrets and 872 in the generated log, and at 64 KiB still 1 and 12. Overlapping windows removed most of these leaks but not all: at 1 KiB they still leaked 2 RedactionBench values, 1 presidio-research value and 2 secrets even with $o = 8192$. All seven residual misses come from detectors that read a cue to their left (a key name, a URL-credential prefix, an account label) that lay in the previous window; a longer o, which only extends a window to the right, cannot recover them, whereas SafeStream keeps exactly this context as its read-only prefix P. On Loghub, naive chunking missed 388 of 18,329 reference detections at 1 KiB and overlapping windows still missed 3. SafeStream's detection set was identical to the whole-document reference in all 100 pattern-tier runs (five streams, four chunk sizes, five chunkings), and with the spaCy tier on presidio-research, 66 RedactionBench documents, three Loghub systems and all 127 TAB judgments (where naive chunking missed 15 detections at 4 KiB); for spaCy this is an empirical check, not a proof (Section 5.1).

The condition applied to someone else's detectors

Nothing in Eq. (1) is specific to SafeStream's detectors, so we wrapped another tool's pipeline: Presidio's pattern recognisers, with overlapping spans merged as a redactor masks them, declared bounds $L = 2048$ and $C_{\text{left}} = C_{\text{right}} = 64$ (regex lookarounds read only a few characters), and SafeStream's engine at $w = 2112$ (script `benchmarks/wrap_presidio.py`; Online Resource 1, Section S16). Chunked Presidio leaked as SafeStream's tiers do without the engine: at 1 KiB, naive chunking left 204 RedactionBench gold values that whole-document Presidio redacts, and overlapping windows still missed up to 1,117 detections even with a 2,112-character overlap. The wrapped engine reproduced whole-document Presidio in every run on presidio-research but in none on RedactionBench or TAB, and the comparison located the cause in a single pattern. Presidio's low-confidence Indian PAN pattern begins with the lookaheads `(?=.*?[a-zA-Z])(?=.*?[0-9]{4})` and is compiled with `DOTALL`, so a ten-character word is reported whenever four digits appear anywhere later in the

Table 3 Leaks caused by chunking alone. Each cell gives missed reference detections / gold spans leaked by chunking (fully redacted by the whole-document pass, not by the variant); Loghub has no gold, so its cells give missed detections only. SafeStream: runs whose detection set is identical to the whole-document reference, out of the random chunkings tried. Overlapping windows keep the detections that start in their own chunk; o is the window overlap in characters. The generated log is synthetic and dense (56,153 gold values in 4.2 MB). Selected rows; Online Resource 1 (Table S1) gives every corpus at 1, 4 and 64 KiB.

Corpus (pipeline), chunk c	Naive	$o = 256$	$o = 8192$	SafeStream
secrets (pattern), 1 KiB	11 / 5	4 / 3	2 / 2	5 of 5
presidio-research (pattern), 1 KiB	5 / 7	1 / 1	1 / 1	5 of 5
RedactionBench (pattern), 1 KiB	74 / 237	3 / 2	3 / 2	5 of 5
RedactionBench (pattern), 64 KiB	1 / 1	0 / 0	0 / 0	5 of 5
Generated log (pattern), 1 KiB	957 / 872	0 / 0	0 / 0	5 of 5
RedactionBench, 66 docs (+spaCy), 4 KiB	20 / 10	2 / 0	2 / 0	2 of 2
Loghub, 16 real systems (pattern), 1 KiB	388	3	3	5 of 5
Loghub, 16 real systems (pattern), 64 KiB	9	0	0	5 of 5
Loghub, 3 systems (+spaCy), 4 KiB	18	4	4	2 of 2
TAB, 127 judgments (+spaCy), 4 KiB	15 / 2	3 / 0	3 / 0	2 of 2

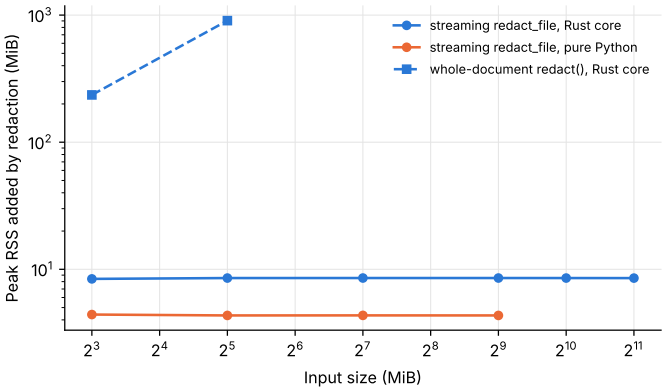


Fig. 3 Peak resident memory added by redaction (peak RSS minus RSS after imports, MiB) against input size, log scales, each point a fresh process. Streaming `redact_file` stays flat from 8 MiB to 2 GiB (Rust core) and to 512 MiB (pure Python); whole-document `redact()` grows with the input.

text: its right context is unbounded, and no overlap can make chunked output equal whole-document output. With that one pattern removed, the wrapped engine was identical to whole-document Presidio in all 12 runs (three corpora, 1 KiB and 4 KiB, two random chunkings), while naive chunking still leaked 145 RedactionBench gold values. The declared bounds are therefore not a property of one implementation: they make an existing detector pipeline safe to stream, and checking them exposes the detectors that cannot be.

7.2 RQ2: memory and throughput

Figure 3 plots peak resident memory against input size. Each point is a fresh process redacting a generated log file dense with personal data and credentials. With the Rust core, streaming `redact_file` added between 8.4 and 8.54 MiB at every size from

Table 4 Test-split results with 95% document-level bootstrap intervals. SafeStream rows are after the entropy fix of Section 7.4, which changed only the secrets results. FR: fully redacted rate; FA: false-alarm document rate.

Corpus / system	Char P	Char R	Char F1	FR	FA
presidio-research (750 documents)					
SafeStream pattern	.995 [.992, .998]	.154 [.133, .176]	.267 [.235, .299]	.115 [.098, .134]	.000
SafeStream +dates	.996 [.993, .998]	.169 [.146, .192]	.290 [.255, .322]	.129 [.110, .150]	.000
SafeStream +spaCy (sm)	.898 [.877, .918]	.576 [.547, .606]	.702 [.679, .725]	.538 [.509, .568]	.079
Presidio regex	.948 [.919, .974]	.175 [.151, .202]	.296 [.260, .333]	.123 [.105, .145]	.000
Presidio sm	.871 [.848, .893]	.518 [.491, .546]	.650 [.626, .673]	.506 [.479, .536]	.048
Presidio lg	.878 [.856, .900]	.535 [.507, .563]	.665 [.642, .688]	.562 [.534, .590]	.000
secrets (313 documents)					
SafeStream pattern	.857 [.705, .946]	.829 [.740, .887]	.842 [.736, .906]	.437 [.384, .498]	.010
SafeStream +spaCy (sm)	.757 [.604, .858]	.845 [.762, .897]	.799 [.683, .870]	.488 [.435, .548]	.340
gitleaks	.837 [.630, .962]	.473 [.299, .679]	.604 [.430, .765]	.311 [.253, .375]	.000
TruffleHog	.803 [.719, .878]	.189 [.117, .308]	.306 [.204, .446]	.270 [.204, .340]	.000
Presidio regex	.282 [.211, .356]	.105 [.063, .177]	.153 [.098, .225]	.126 [.083, .168]	.392
Presidio sm	.251 [.191, .315]	.119 [.072, .197]	.161 [.107, .232]	.160 [.112, .206]	.649
Presidio lg	.308 [.238, .381]	.131 [.082, .214]	.184 [.124, .262]	.164 [.116, .212]	.433

8 MiB to 2 GiB, a 256-fold range. The pure-Python backend added between 4.3 and 4.4 MiB from 8 to 512 MiB; it was not run at larger sizes because of its run time (726 s at 512 MiB). Whole-document `redact()` on the same generated inputs, measured the same way, added 235.5 MiB at 8 MiB and 904.6 MiB at 32 MiB, about 28 times the input size, because the input string, the detections and the output are held at once; at 32 MiB the two modes therefore differ by a factor of about 106 on identical input. A 128 MiB whole-document run was stopped after 20 minutes without completing and is not reported. These measurements are consistent with Theorem 3 but cover one generated input shape on one machine, and only two whole-document sizes.

On RedactionBench, whose documents average 13.8 kB, the Rust core processed 10.4 ± 1.3 MB/s with one thread, the pure-Python reference 1.23 ± 0.15 MB/s and Presidio's pattern recognisers 0.069 MB/s; with a statistical model in the loop every system ran below 0.05 MB/s. On a 256 MB generated log, `redact_file` with the Rust core ran at 10.5 ± 0.5, 20.6 ± 0.5 and 41.1 ± 1.4 MB/s with 1, 2 and 4 worker processes, a 3.9-fold speed-up on 4 vCPUs. Online Resource 1 (Table S2 and Fig. S1) gives every corpus and system. These figures come from one virtual machine and should be read as relative rather than absolute.

7.3 RQ3 and RQ4: detection on held-out data

Table 4 gives the test-split results (plotted in Online Resource 1, Fig. S2); The planned differences are tested under clustering in Online Resource 1 (Table S9).

Table 5 Real credentials (CredData): 63,706 labelled line ranges from 337 repositories, values obfuscated by the dataset's builder. FR: fully redacted rate over the 14,626 credential values; FA: share of the 49,517 non-credential ranges with a detection. 95% intervals under two-stage resampling by repository. SafeStream rows: pattern tiers at the evaluated commit, and the current code's default profile.

System	Char P	Char R	Char F1	FR	FA
SafeStream pattern (evaluated)	.354 [.150, .625]	.817	.494 [.249, .720]	.548 [.313, .739]	.178 [.100, .271]
SafeStream default (current)	.343 [.146, .634]	.842	.487 [.246, .735]	.717 [.545, .834]	.328 [.291, .360]
gitleaks	.879 [.714, .945]	.630	.734 [.431, .840]	.398 [.149, .632]	.023 [.003, .057]
TruffleHog	.207	.004	.008	.007	.006

Credentials (RQ3)

On secrets, SafeStream's pattern tiers fully redacted 0.437 of test secrets, against 0.311 for gitleaks and 0.270 for TruffleHog (paired document-level differences +0.128 [+0.073, +0.183] and +0.167 [+0.079, +0.250]); precision did not differ significantly. Under resampling by rule (Online Resource 1, Table S9), the advantage over gitleaks remains in fully redacted rate (two-stage +0.125 [+0.025, +0.209]) and character F1 (+0.226 [+0.011, +0.538]); against TruffleHog only the character-F1 advantage remains (+0.530 [+0.192, +0.742]). SafeStream flagged one of 97 generated non-secrets, neither scanner any. More than half of the test secrets (0.563) were not fully redacted by SafeStream either, mostly vendor formats it has no pattern for; adding spaCy raised its false-alarm rate to 0.340.

Scanner sensitivity

The gold labels are Nosey Parker's rule examples, and both scanners filter example-like strings, so we re-ran them with their false-positive filters disabled and their rules unchanged (Online Resource 1, Section S4). gitleaks gained in character F1 (0.604 to 0.805) but little in fully redacted rate (0.311 to 0.328): SafeStream's fully-redacted-rate advantage remains robust (+0.111; two-stage [+0.008, +0.193]), its character-F1 advantage does not survive rule clustering, and TruffleHog changed little. SafeStream's credential patterns were not developed on this corpus; the only change made using it was the suppression rule of Section 7.4, designed on the development half.

Real credentials (CredData)

The secrets corpus holds rule authors' examples; CredData holds credentials found in real files and labelled by people (Table 5; protocol committed before any run). The planned comparisons show a trade-off rather than a winner. SafeStream's pattern tiers fully redacted more real credentials than gitleaks, 0.548 against 0.398 (+0.150 [+0.142, +0.157]; two-stage by repository +0.147 [+0.085, +0.273]), so the credential-recall margin of the synthetic evaluation holds on real data and survives clustering. They also raised a false alarm on more of the lines that people judged not to hold a credential, 0.178 against 0.023 (+0.155; two-stage [+0.095, +0.217]), and gitleaks' character F1 was higher with lines resampled independently (−0.237 [−0.298, −0.185] for SafeStream) though not robustly so under clustering ([−0.410, +0.011]). The

current code's default profile fully redacted 0.717 of credentials, chiefly because its structured-field tier finds passwords in configuration syntax (fully redacted rate for passwords 0.88, against 0.19 at the evaluated commit and 0.05 for gitleaks), at a false-alarm rate of 0.328. By category, SafeStream led on keys, authorisation headers, URL credentials and nonces, and gitleaks on private keys, tokens and generic secrets (Online Resource 1, Table S7). TruffleHog redacted almost nothing, plausibly because its detectors key on vendor formats that the shape-preserving obfuscation does not keep; we did not verify this.

Where the false alarms come from (descriptive; Online Resource 1, Section S15). The entropy tier, which supplies most of SafeStream's credential recall, also causes most of its false alarms: without it the pattern tiers fully redacted 0.089 of credentials at a false-alarm rate of 0.075, and 58% of the flagged non-credential lines carried only entropy-tier detections. SafeStream's confidences do not offer a gitleaks-like operating point: raising the threshold from 0.60 to 0.65 lowers the false-alarm rate to 0.070 but the fully redacted rate to 0.089. CredData's non-credential lines are of two kinds, values judged not to be credentials (`F`, 45,442 ranges) and test values, examples and placeholders (`X`, 3,977). The evaluated pattern tiers flagged the two at similar rates (0.178 and 0.167); the current code flagged 0.85 of the `X` lines, which a redactor that fails closed arguably should mask. The trade-off is a design choice: a redactor that over-redacts loses text, one that under-redacts leaks a secret, while for an alerting scanner the same gap would mean many more alerts to review, which is why gitleaks trades recall for precision. One correction was made after scoring and is disclosed in the protocol: 1,091 ranges labelled as credentials but without a value position had first been counted as non-credential; the protocol counts them as neither, they were removed, and every system's false-alarm rate fell (gold is unchanged).

Personal data (RQ4)

On presidio-research, the pattern tiers were precise (character precision 0.995) but recalled only 0.154 of sensitive characters, because most gold spans are names, locations and organisations. With the same spaCy model, SafeStream +spaCy exceeded Presidio sm in character F1 by +0.051 [+0.027, +0.076], robust to template clustering (two-stage +0.052 [+0.013, +0.093]); its fully-redacted-rate advantage and its F1 advantage over Presidio lg are not robust. This corpus is synthetic, and Section 7.8 shows that the result does not carry over to real judgments.

7.4 False alarms on high-entropy non-secrets

Before the suppression rule of Section 4.1, the entropy tier flagged 83 of the 103 generated non-secrets in the development split. The rule, and the secret-key veto that keeps a UUID-shaped access key detected, were designed on the development split. On the test split the false-alarm rate fell from 0.794 [0.714, 0.872] to 0.010 [0.000, 0.035], character precision rose from 0.589 to 0.857, and recall was unchanged. The test half had been scored once with the pre-fix code before the fix was made; both scores are reported. Because the gold-free documents in both halves come from the same generator, the test improvement is an optimistic estimate of the fix's effect on real non-secrets (details in Online Resource 1, Section S5).

Table 6 RedactionBench test half (100 documents), with 95% document-level bootstrap intervals. *Exposed*: current code, whose rules were written after inspecting all 200 documents. *Pre-exposure*: the last commit before the benchmark entered the repository. *Dev-selected*: configuration chosen on the development half by a fixed rule (maximum development character F1) and scored on the test half once.

System	Char F1	Mandatory recall	No-leak docs
Pre-exposure, pattern	.518 [.434, .593]	.589 [.476, .696]	.074 [.022, .129]
Pre-exposure, +spaCy	.472 [.395, .547]	.674 [.566, .779]	.137 [.073, .208]
Pre-exposure, dev-selected	.469 [.394, .543]	.676 [.568, .781]	.137 [.073, .208]
Exposed, pattern	.579 [.490, .662]	.682 [.578, .787]	.084 [.032, .144]
Exposed, +spaCy	.512 [.429, .590]	.761 [.657, .860]	.179 [.105, .263]
Exposed, dev-selected	.508 [.427, .588]	.762 [.658, .862]	.179 [.105, .263]
gitleaks	.214 [.130, .314]	.247 [.140, .372]	.021 [.000, .053]
TruffleHog	.030 [.017, .048]	.024 [.013, .040]	.000
Presidio regex	.323 [.250, .393]	.331 [.227, .422]	.011 [.000, .033]
Presidio sm	.336 [.265, .402]	.416 [.326, .500]	.084 [.032, .146]
Presidio lg	.344 [.277, .409]	.420 [.333, .499]	.105 [.052, .174]

7.5 RQ5: document-level protection and contamination

Table 6 separates the contaminated and uncontaminated results. Rules written after inspecting the benchmark raised test mandatory recall of the pattern tiers by +0.091 [+0.044, +0.163] and character F1 by +0.060 [+0.029, +0.108] relative to the pre-exposure code (paired), and both gains are robust to domain clustering. Because those rules were written while the test documents were visible, this gain is an upper bound on what they contribute to unseen data. One of them, the label-anchored tier, also improved presidio-research, a corpus it was never tuned on (Section 7.10), which is evidence that it generalises. Development-set selection chose the same configuration for both code versions (spaCy on, entropy on, confidence threshold 0.3).

Pre-exposure code against the baselines

The pre-exposure code never saw the benchmark, so it gives the uncontaminated comparison (Online Resource 1, Table S9). With spaCy, its mandatory recall exceeded Presidio lg and sm by +0.262 [+0.091, +0.436] and +0.265 [+0.093, +0.441], robust to domain resampling; its character-F1 advantage is robust against Presidio sm only, and the pattern tiers' F1 advantage over Presidio does not survive domain clustering. Against the secret scanners every pre-exposure configuration was ahead under all three schemes; no-leak rates did not differ significantly from Presidio.

No-leak documents

The no-leak document rate was low for every system: at most 0.179 [0.105, 0.263] for SafeStream (+spaCy, exposed code), 0.137 for the pre-exposure code with spaCy, 0.084 for the pattern tiers alone, and at most 0.105 for Presidio. In other words, for at least 82% of RedactionBench test documents, every configuration we evaluated left at least one must-redact character in the clear. This metric is deliberately strict. Mandatory recall measures the share of must-redact characters hidden; the no-leak rate requires all of them in a document to be hidden, so a single missed character fails

the document. RedactionBench documents are dense: the test half averages about 256 gold spans per document (Table 2), of which 84 on average (median 25) are mandatory.

Where the leaks are. Per-document predictions of five systems (Online Resource 1, Section S6) show that the leaks are whole missed values rather than boundary errors (4.8% of leaked mandatory characters with spaCy lie in partly redacted spans), that they are concentrated (fixing one or two spans per document would raise the no-leak rate with spaCy only to 0.305), and that they are mostly names without a named-entity model. Even the union of all five systems' redactions would leave 69 of 95 documents leaking (no-leak rate 0.274), so the low document-level rate is a property of the corpus and of current detectors in general, not only of SafeStream. Nor is it caused by streaming: by Theorem 1 the streamed output equals the whole-document output, so these leaks are detection misses that a whole-document run would also make.

7.6 RQ6: detection on real system logs

On real `sshd` lines (all systems in Online Resource 1, Table S10), every system with an IP pattern, SafeStream and all three Presidio configurations, redacted all 1,349 IP addresses. No system redacted user names: SafeStream's pattern tiers redacted none of 1,142, and the best configuration, SafeStream with spaCy, redacted 21. User names in phrases such as `Invalid user webmaster from` carry no shape or label that a pattern tier keys on, and the statistical models rarely tag them. This is the clearest gap that real data exposed. Because user names are nearly half the gold, the fully redacted rate is 0.54–0.55 for every IP-capable system, and only 38–40% of lines with gold were fully protected.

The systems differed in what else they redacted. Presidio's statistical configurations tagged the syslog timestamp of almost every line as a date and flagged 99–100% of the lines without gold, against none for SafeStream's pattern tiers, so SafeStream's character F1 exceeded Presidio sm and lg by +0.291 and +0.208, both robust to template clustering (two-stage [+0.205, +0.351] and [+0.130, +0.275]); against Presidio's pattern recognisers the difference (+0.038) does not survive clustering. An independent audit of the labels against the values each slot holds (`benchmarks/loghub_label_audit.py`; Online Resource 1, Section S17) found one defect in how the frozen labels were applied: in 383 lines Loghub's parser leaves a trailing space after the remote address, so the address was not recognised as an IP and was left out of the gold; in another 90 the host is a reverse-DNS name embedding the address, which the rule excludes. Every IP-capable system redacted both, so their precision is understated. With the whitespace handled, character F1 rises for every such system by a similar amount (SafeStream 0.757 to 0.879, Presidio regex 0.719 to 0.838, Presidio lg 0.549 to 0.654), and no comparison changes direction; the frozen labels remain the primary result.

7.7 A user-name tier for authentication logs

In authentication logs a user name is identified not by its shape but by the fixed text the logging program writes around it, so this gap can be closed by rules. We added

Table 7 Held-out real Linux logs (2,000 lines; Loghub). Labels were drafted after the user-name tier was committed and frozen before scoring. IP and user-name columns give the gold values fully redacted by any detection; FA is over the 407 lines without gold. The upper block is the evaluated commit, as in the rest of the paper; the lower block is the current code with and without the user-name tier.

System	Char P	Char R	Char F1	IP addresses	User names	FA
SafeStream pattern	.956	.866	.909	1,258 / 1,258	0 / 618	.005
SafeStream +spaCy (sm)	.532	.866	.659	1,258 / 1,258	0 / 618	.452
Presidio regex	.403	.866	.550	1,258 / 1,258	0 / 618	.251
Presidio sm	.270	.866	.412	1,258 / 1,258	0 / 618	.526
Presidio lg	.285	.866	.429	1,258 / 1,258	0 / 618	.686
gitleaks	.000	.000	.000	0 / 1,258	0 / 618	.000
TruffleHog	.000	.000	.000	0 / 1,258	0 / 618	.000
Current code, default	.959	.944	.952	1,258 / 1,258	372 / 618	.005
Current code + user-name tier	.961	1.000	.980	1,258 / 1,258	618 / 618	.005

an opt-in tier (`use_log_usernames`) of 14 patterns anchored on the documented message formats of `sshd` (`Invalid user NAME from`), Linux-PAM (`user=NAME`, `session opened for user NAME`), `su`, `sudo` and `login`; only the name is redacted, and its declared bounds ($L = 64$, $C_{\text{left}} = 96$, $C_{\text{right}} = 320$) leave $w^\star$ at 8192. Because the current default already reads `user=NAME`, we compare the current code with and without the tier.

On the held-out Linux logs (Table 7) the tier redacted all 618 user names, every line with gold was fully protected (0.846 without the tier, 0.612 for every IP-capable evaluated configuration), and the false-alarm rate was unchanged. The gain over the current default (+0.131 [+0.115, +0.147] in fully redacted rate) is not robust to template clustering ([+0.000, +0.771]), because all user names occur in 5 of the 118 templates; against every Presidio configuration the tier's character-F1 advantage is (at least +0.429 [+0.338, +0.629]).

On the OpenSSH lines, whose templates had been read while drafting their labels, the tier redacted 1,127 of 1,142 user names (386 without it); the 15 misses are the literal name `user` and three names logged after a double space. Both corpora come from two machines, so this is evidence for authentication logs, not for user names in general.

7.8 RQ7: personal data in real court judgments

On TAB's test split (full results in Online Resource 1, Tables S11 and S5) the evaluated configurations did poorly, reversing the synthetic result of Section 7: SafeStream +spaCy had lower character F1 than Presidio sm, −0.213 [−0.243, −0.181], and than Presidio lg, −0.225 [−0.255, −0.193]. Most of the gap is dates, 44.7% of TAB's gold characters, which Presidio redacted (0.98) and SafeStream's named-entity tier, mapping only person, organisation and place labels, did not (0.01). Direct identifiers, 1,011 person-name and 587 code mentions such as application numbers (`36110/97`), were rarely protected by any system: entity recall on them was at most 0.077, and in fewer than 1% of document–annotator pairs was every direct identifier redacted. An entity counts only if all its mentions are redacted, so a name whose title ("Mr") is left out at one mention is unprotected, and application numbers carry no label or checksum that

any evaluated tier keys on. The pattern tiers alone recalled almost nothing (0.004); the secret scanners flagged nothing.

7.9 A free-text configuration, confirmed on held-out judgments

The TAB failures have identifiable causes, so we built a configuration for prose and tested it the way the rest of the paper tests rules: designed on development data, frozen, then scored once on data no one had looked at. The protocol, including the systems, metrics and planned comparisons, was committed before any system was run on TAB's development or training split (commit `f2be354`). TAB's test split had already been seen and is reported only as a secondary, exposed result. The configuration, `Redactor(profile="prose", use_ner=True)` at commit `9c10975`, adds three general mechanisms, all off in the default profile and all within the declared bounds, so $w^\star$ stays 8192:

- the prose profile's date tier, plus bare years, which the date tier does not match;
- a prose-values tier: sums of money with their currency and number/year references such as case and application numbers (`47676/99`), each matched on word boundaries;
- a prose mode of the spaCy tier that keeps a person's title with the name (spaCy leaves out "Mr" in "Mr Eric Oldham", so the gold mention was only partly redacted) and skips generic institution words such as "the Court" or "Government".

One candidate rule, treating all-capital words as section headings, was tried on the development split and removed because it would also drop acronym organisations, which can identify. No rule names a TAB value.

On the held-out judgments (Table 8) the free-text configuration was ahead of Presidio lg on every planned comparison: character F1 by +0.131 [+0.122, +0.139], character recall by +0.177 [+0.166, +0.187], entity recall on direct identifiers by +0.764 [+0.744, +0.784] and on quasi identifiers by +0.112 [+0.104, +0.122]. Against Presidio sm every margin was larger (character F1 +0.141 [+0.133, +0.149]). It also had higher precision than either (0.568 against 0.465 for lg). The largest change is in direct identifiers: in 78.7% of document–annotator pairs every direct identifier was fully redacted, against at most 0.9% for every other system, because titles now stay with names and application numbers are caught (code recall 0.91 against 0.04 for Presidio lg; by type in Online Resource 1, Table S6). On the development split the configuration reached character F1 0.742; the held-out value, 0.670, is the one to quote. On TAB's exposed test split it reached 0.732 [0.715, 0.748], against 0.605 for Presidio lg. Its streamed output was identical to its whole-document output on held-out judgments in every chunking tried (941,155 characters, the most spaCy accepts in one call; 4 KiB and 64 KiB, two chunkings each), where naive 4 KiB chunking leaked 22 gold mentions and overlapping windows one.

The configuration was designed on judgments from the same court as that held-out set, so we also ran it, unchanged and under a protocol fixed beforehand, on a second genre: 553 annotated Wikipedia biographies (lower block of Table 8). There it again had higher character F1 than Presidio lg, +0.129 [+0.115, +0.144], higher recall (+0.162 [+0.143, +0.182]), higher precision (0.827 against 0.753) and higher

Table 8 The frozen free-text configuration on held-out data: TAB's training split (1,014 judgments, 1,112 document–annotator pairs), never inspected or scored before the configuration was frozen, and a second genre, Wikipedia biographies (553 summaries, 573 pairs), run with the configuration unchanged. SafeStream free-text: `profile="prose"` with spaCy, frozen before this run. ER: entity recall, the share of entities all of whose mentions are fully redacted; *Dir. prot.*: share of document–annotator pairs in which every direct identifier is fully redacted. 95% document-level intervals.

System	Char P	Char R	Char F1	ER direct	ER quasi	Dir. prot.
TAB held-out judgments						
SafeStream free-text	.568	.817	.670 [.660, .679]	.892 [.878, .906]	.805 [.797, .813]	.787
SafeStream +spaCy (evaluated)	.368	.360	.364 [.353, .374]	.120 [.107, .134]	.156 [.150, .163]	.004
Presidio regex	.067	.038	.048 [.044, .052]	.009 [.005, .014]	.006 [.005, .007]	.001
Presidio sm	.458	.625	.528 [.519, .537]	.112 [.099, .127]	.672 [.661, .683]	.009
Presidio lg	.465	.641	.539 [.530, .548]	.129 [.113, .144]	.693 [.682, .703]	.006
Wikipedia biographies (second genre)						
SafeStream free-text	.827	.710	.764 [.752, .776]	.660 [.625, .694]	.617 [.600, .634]	.585
SafeStream +spaCy (evaluated)	.845	.535	.655 [.641, .671]	.516 [.483, .551]	.360 [.341, .377]	.494
Presidio regex	.425	.048	.086 [.076, .096]	.001 [.000, .004]	.017 [.013, .021]	.000
Presidio sm	.747	.515	.610 [.596, .624]	.559 [.526, .596]	.481 [.465, .497]	.461
Presidio lg	.753	.548	.634 [.620, .650]	.689 [.656, .722]	.505 [.489, .522]	.618

quasi-identifier entity recall (+0.113 [+0.095, +0.130]), and every margin over Presidio sm was positive, direct identifiers included (+0.101 [+0.076, +0.127]). The direct-identifier gain did not carry over against Presidio lg (−0.029 [−0.061, +0.004]): biographies carry no application numbers, and their direct identifiers are names, where the larger spaCy model behind Presidio lg holds its own.

Three limits apply. The configuration has been tested on two genres of formal English prose, not on free text in general. Its precision is moderate: most remaining false alarms are names of courts and states that TAB's annotators did not mark. And one direct identifier in five document–annotator pairs still leaked somewhere, so the configuration reduces exposure substantially without making redacted judgments safe to publish on its strength alone.

7.10 Ablation

An ablation on the test splits (Online Resource 1, Table S4; document-level intervals, not planned) shows that the entropy tier carried most of the credential recall: without it, character F1 on secrets fell from 0.842 to 0.383 (+0.455 [0.242, 0.641]). The label-anchored tier raised character F1 on presidio-research by +0.030 [0.019, 0.044], a corpus it was never tuned on. The contextual scorer's own detections changed no result, and the named-entity union rule raised recall by small, mostly significant amounts (at most +0.013) at no significant cost in F1 on secrets or RedactionBench.

8 Discussion

8.1 What the results establish, and what they do not

Established under stated assumptions: chunked and parallel processing introduce no detection difference relative to the whole-document pipeline, for detectors that respect their declared bounds (Section 5). *Measured:* leaks caused by naive and overlapping chunking on every corpus, including real system logs and court judgments, and none with SafeStream; flat working memory across a 256-fold range of input sizes; near-linear scaling to four cores; the robust detection margins of Section 7; and, on real logs, complete IP-address redaction, user names redacted by the opt-in tier on held-out data, and no false alarms from the pattern tiers where Presidio's statistical configurations flagged most lines; on real court judgments, lower personal-data recall than Presidio for the evaluated configurations, and, for a free-text configuration designed on development data, higher character F1 and far higher direct-identifier protection than Presidio on 1,014 held-out judgments, and higher character F1 on a second genre, Wikipedia biographies; on 14,626 real credentials, more fully redacted than gitleaks, with more false alarms on lines people judged harmless. *Not established:* effectiveness on credentials outside CredData's candidate set and file types; document-level protection adequate for public release; robustness to adversarial obfuscation.

8.2 When is the approach useful despite a low no-leak rate?

Bounded memory and equivalence do not make redaction complete; they remove one failure mode, the leak introduced by chunking itself, and make large inputs processable at all. The evidence supports three uses and argues against a fourth. (i) *Defence in depth for structured secrets in machine-generated text*, where the pattern tiers have their highest coverage and the guarantee ensures that splitting a multi-gigabyte log does not lower it; even here some credentials are missed, so the tool reduces exposure rather than eliminating it. (ii) *A first pass before human review*, in logs and, with the free-text configuration, in formal prose. (iii) *Inputs that cannot otherwise be processed*: whole-document redaction needed about 28 times the input size in memory. (iv) It is *not* supported as the sole control for publishing free text: no-leak rates stayed below 0.18 on RedactionBench, and a direct identifier still leaked somewhere in about one held-out TAB document–annotator pair in five. A deployment should set its own acceptance criteria, such as per-category recall on an authorised sample of its artefacts, zero tolerated leaks for named credential types and periodic re-measurement; none of these was measured here.

8.3 Threats to validity

External validity. presidio-research is synthetic, and its result does not carry over to TAB (Section 7.8); the secrets positives are rule-author examples; RedactionBench is contaminated; all corpora are English. The real-log labels come from templates checked by one author rather than double annotation; an independent audit against the 4,000 lines' slot values found one defect in how they were applied, whose effect we report (Section 7.6), and a blind second-annotation sheet with an agreement tool

is released. The user-name tier's OpenSSH result is exposed. CredData's labels cover only candidates that other tools proposed, gitleaks and TruffleHog among them; its values are obfuscated; each document is one line range rather than a whole file; and one loader correction, disclosed with its effect, was made after scoring. The free-text configuration was designed on TAB and tested on one other genre, where its direct-identifier advantage did not hold against Presidio lg; TAB's test split motivated it and is reported as exposed. *Construct validity.* The secrets gold spans are Nosey Parker's capture groups, so credentials adjacent to a labelled secret count as false positives; RedactionBench's contextual labels have low human agreement [33]. *Internal validity.* The secrets test half was scored before and after one fix (Section 7.4). *Statistical validity.* Eleven RedactionBench domains are few clusters for a bootstrap; the ablation intervals are document-level only; no multiplicity correction was applied. *Comparison fairness.* With their filters disabled, gitleaks closed most of the character-F1 gap on the secrets corpus (Section 7.3), so that margin is not claimed. *Environment.* All runs used one virtual machine.

8.4 Evaluation plan for real-world evidence

TAB now provides real free text with independent human gold; CredData provides real credentials; the most valuable next evidence is free text from other domains and real credential-bearing text found by a process independent of existing scanners with clear provenance, such as public continuous-integration logs whose licences permit research use or logs from an organisation that authorises the study, annotated by two people with agreement reported. Safeguards: never test discovered credentials against live services, keep raw text encrypted and limited to annotators, report live secrets to their owners, and publish only annotations, hashes and aggregate results. The repository contains a collection plan, annotation guidelines and a scoring pipeline for such a corpus (`benchmarks/realworld/`). Transformer-based detectors would additionally need an empirical streamed-versus-whole-document check, because the theorems do not cover a model whose output depends on its whole input window.

9 Conclusion

Chunked redaction need not trade correctness for scale. With detector-declared bounds on span length and context, an overlap that covers every detector's span plus its right context makes the streamed result equal to the whole-document result, a seam cross-check makes the parallel result equal to the sequential one, and the working buffer does not grow with the input. Naive and overlapping chunking leaked values that a whole-document pass redacts, on test corpora and real logs alike, while SafeStream's output matched the whole-document output in every chunking tried; memory stayed flat from 8 MiB to 2 GiB. These properties remove chunking as a source of leaks; they do not make detection complete. Two detection margins survived cluster-robust resampling and the scanner sensitivity run: more credentials fully redacted than gitleaks, which held on 14,626 real credentials (0.55 against 0.40) at a higher false-alarm rate, and, on a synthetic corpus, higher character F1 than Presidio with the same spaCy model. On real court judgments with independent human gold

that second margin reversed for the evaluated configuration; a free-text configuration designed on development judgments and frozen before a held-out run then reached character F1 0.67 against 0.54 for Presidio and fully protected the direct identifiers of 79% of held-out document–annotator pairs against at most 1%; unchanged, it also led on character F1 on Wikipedia biographies. In real logs every IP-capable system redacted all IP addresses but none redacted user names, a gap that an opt-in tier built from documented log formats closed on held-out Linux logs. Other margins did not survive, and fewer than one RedactionBench document in five was fully protected by any configuration. The system is therefore suited to reducing exposure in large machine-generated artefacts as one layer among several, and, with the free-text configuration, to a strong first pass over formal prose that a person then reviews; it is not suited to certifying documents for public release on its own. The next steps are free text beyond formal prose and credentials found by a process independent of existing scanners (Section 8.4).

Acknowledgements. The authors gratefully acknowledge Sri Mata Amritanandamayi Devi (Amma), Chancellor of Amrita Vishwa Vidyapeetham, for her inspiration and for providing financial support towards the Article Processing Charges (APC) of this publication. The authors also thank the maintainers of Microsoft Presidio, Nosey Parker and RedactionBench for making their software and benchmark datasets publicly available.

Declarations

- **Funding.** The authors gratefully acknowledge the financial support provided by Sri Mata Amritanandamayi Devi (Amma), Chancellor of Amrita Vishwa Vidyapeetham, towards the Article Processing Charges (APC) of this publication.
- **Competing interests.** The authors declare no competing financial or non-financial interests.
- **Ethics approval and consent to participate.** Not applicable: no human participants were involved and the authors collected no data. presidio-research is synthetic, the secrets corpus consists of example strings from an open-source rule set together with generated negatives, RedactionBench is a public dataset released by its authors under CC BY 4.0, which they describe as seeded from real-world sources; the Loghub logs, the TAB court judgments and the Wikipedia biographies are public research datasets used under their authors' terms. CredData was built with its authors' script, which fetches public repositories and replaces every real credential with a random value of the same shape; the raw checkouts were deleted as soon as it had done so, and only obfuscated values were evaluated. All processing was local; no excerpt of any document and no credential value is published here, and no credential was sent to a live service (TruffleHog ran without verification).
- **Consent for publication.** Not applicable.
- **Data availability.** presidio-research, Nosey Parker and Loghub are public (Loghub is free for research use with citation of [34]); RedactionBench is available from its authors under CC BY 4.0; TAB [17] is public under the MIT licence; the annotated Wikipedia biographies [35] are public from their authors; CredData [36] is rebuilt

from public repositories with its authors' script. The template-slot labels for the Loghub corpora are in the repository. Generated non-secrets are produced by the released code with a fixed seed. Split manifests, raw results and per-document predictions (character offsets only, no corpus text) are in the public repository under `research/results/`.

- **Code availability.** Source code, the evaluation harness, the analysis scripts (for example the cluster bootstrap, the TAB scorer and the second-annotator agreement tool) and the pre-registered protocols for the real-data evaluations are in the `benchmarks/` directory of the public repository, https://github.com/MounishSenisetty/SafeStream-Redactor.
- **Author contributions.** Mounish Senisetty: conceptualisation, methodology, software architecture, formal analysis, validation, investigation, data curation, visualisation, writing (original draft) and writing (review and editing). Ashrith Reddy N: software implementation, validation, investigation and writing (review and editing). Sandhiya Kennedy: software implementation, validation, investigation and writing (review and editing). Parvathy R: supervision and writing (review and editing). All authors read and approved the final manuscript.
- **Use of generative AI.** During the preparation of this work the authors used the large language model Claude (Anthropic) to help implement evaluation scripts, draft early manuscript text, convert LaTeX markup to the journal template, edit language and check bibliographic references. After using this tool, the authors reviewed and revised all code, experimental results and text, and take full responsibility for the content.

Appendix A Proofs of Theorems 2 and 3

Online Resource 1 (Table S8) maps each step below to the code that discharges it and the tests that corroborate it. Throughout, positions are absolute offsets in T, the conditions of Theorem 1 hold unless stated otherwise, and $L_{\max} = \max_d L_d \le w$.

A.1 A streaming engine started mid-text

The parallel engine runs Algorithm 1 on a substring of T with a read-only left context. Lemma 1 covers that case.

Lemma 1 *Let $X = T[x_0{:}x_1]$ and let the engine start with $P = T[x_0{:}x_0 + h]$ as already-emitted context and stream $T[x_0 + h : x_1]$, where $h = w$, or $h < w$ and $x_0 = 0$. Then it emits exactly the spans of $\Pi(X)$ that start at or after $x_0 + h$.*

Proof The proof of Theorem 1 uses only two properties of P: it holds the $\min(w, p - x_0)$ characters preceding B, and the window ends at the end of the text being processed. Both hold here with X in place of T. Spans of the first window that start inside P are dropped, as in every later pass. □

Lemma 2 (Transfer to the full text) *With $X = T[x_0{:}x_1]$, let s be a span of detector d that belongs to $\Pi(X)$ or to $\Pi(T)$, with $i - C_{\text{left},d} \geq x_0$ or $x_0 = 0$, and $j + C_{\text{right},d} \leq x_1$ or $x_1 = |T|$. Then $s \in \Pi(X)$ if and only if $s \in \Pi(T)$.*

Proof Apply A2 with $a = x_0$ and $b = x_1$ in T; in X these positions are its two edges. This gives $\delta = -x_0$ in the notation of A2, and an edge of X that coincides with an edge of T satisfies the edge clause. □

A.2 Proof of Theorem 2

Let the split points be $0 = a_0 < a_1 < \cdots < a_k = |T|$, with $a_{r+1} - a_r \geq 4w$; `plan_ranges` enforces this, and coarsens the split or falls back to one range otherwise. Worker r processes $X_r = T[x_0^r : x_1^r]$ with $x_0^r = \max(0, a_r - 2w)$ and $x_1^r = \min(|T|, a_{r+1} + w)$. Its first w characters, $T[a_r - 2w : a_r - w]$, are read-only context, and streaming starts at $a_r - w$; worker 0 has no context and starts at 0. By Lemma 1, worker r obtains D_r, the spans of $\Pi(X_r)$ that start at or after $a_r - w$.

A *straddler* of a seam a is a span with $i < a < j$. Worker r reports the straddler of a_r (its start seam) and the straddler of a_{r+1} (its end seam) found in D_r, or reports that none exists. It emits $T[\ell_r : u_r]$, where ℓ_r is the end of its start-seam straddler or a_r, and u_r is the end of its end-seam straddler or a_{r+1}. Together with that text it emits the spans of D_r contained in $[\ell_r, u_r)$.

Step 1: in the window that matters, worker r sees $\Pi(T)$. Let s be a span of detector d with $a_r - w \leq i < a_{r+1}$ that belongs to D_r or to $\Pi(T)$. On the left, $i - C_{\text{left},d} \geq a_r - 2w \geq x_0^r$, because $C_{\text{left},d} \leq w$. On the right, $j \leq i + L_d < a_{r+1} + L_d \leq a_{r+1} + w - C_{\text{right},d}$, so $j + C_{\text{right},d} \leq x_1^r$, or $x_1^r = |T|$. By Lemma 2, $s \in D_r \Leftrightarrow s \in \Pi(T)$.

Step 2: both neighbours report the same straddler. A straddler of a_r starts after $a_r - L_d \geq a_r - w$, and a straddler of a_{r+1} starts before a_{r+1}, so both lie in the range of Step 1. Spans that start before $a_r - w$ are dropped as context. Such a span ends at most $L_{\max} \leq w$ characters after its start, that is, at or before a_r, so it cannot cross a_r. Each straddler found by worker r is therefore the unique straddler of $\Pi(T)$ at that seam, if one exists (uniqueness follows from disjointness). Worker $r + 1$ applies Step 1 to its own start seam a_{r+1} and obtains the same span. The reports agree, so the valve passes, and $u_r = \ell_{r+1}$.

Step 3: each part is the sequential redaction of its slice. Every span emitted by worker r lies in $[\ell_r, u_r)$. It starts at or after $\ell_r \geq a_r$, and before a_{r+1}: if there is an end-seam straddler, any span starting at or after a_{r+1} and ending by u_r would overlap it; otherwise $u_r = a_{r+1}$. By Step 1 the emitted spans are exactly the spans of $\Pi(T)$ inside $[\ell_r, u_r)$. No cut point ℓ_r lies strictly inside a span of $\Pi(T)$, because it is either a seam with no straddler or the end of a span of $\Pi(T)$, and $\Pi(T)$ is disjoint. The slices $[\ell_r, u_r)$ therefore partition T without cutting any span. Redaction acts on each span separately (Definition 1), so the concatenated parts equal the redaction of T under $\Pi(T)$. By Theorem 1 that is the sequential output.

Step 4: timing of the cuts. A worker writes text only once the cuts that can affect it are fixed. It decides ℓ_r once its stream has passed $a_r + w$. By then every straddler of a_r, which ends before $a_r + L_{\max}$, has been emitted by the streaming engine. It flushes

only text below the current position minus $2w$. The stream never extends beyond $a_{r+1} + w$, so this text lies below $a_{r+1} - w$, and the end cut $u_r \geq a_{r+1}$ cannot affect it. This proves (i).

Proof of (ii). This part uses no assumption on the detectors. For each seam, the coordinator compares the left worker's end report with the right worker's start report and redacts the whole file sequentially on any mismatch. If all reports match, each u_r equals ℓ_{r+1}, because both are computed from the same reported straddler end. The written parts then cover $[0, |T|)$ as consecutive, non-overlapping slices, and every character of T is emitted exactly once, redacted or not. A non-splittable text encoding, a single worker or a small file also take the sequential path. When the conditions of Theorem 1 fail, agreeing parts can still contain different detections from the sequential output; this is why an overlap below $w^\star$ is refused by default. □

A.3 Proof of Theorem 3

After an emission. The retained buffer is $B[e{:}]$. If no span straddles the planned boundary, $|B| - e = w$. Otherwise $e = i^* > |B| - w - L_d$, so $|B| - e < w + L_{\max}$. In both cases the retained buffer holds fewer than $w + L_{\max} \leq w + m$ characters.

Before a detection pass. Chunks are appended until $|B| \geq w + m$. Before the last of them, $|B| < w + m$, and that chunk adds at most c characters, so a pass starts with $|B| < w + m + c$. The proof of Theorem 1 shows that the pinned branch, which would let B grow further, is never taken when $m \geq w \geq L_{\max}$. The final buffer obeys the same bound. The window $P \cdot B$ adds $|P| \leq w$, which gives $2w + m + c$. The detections of a window are pairwise disjoint and lie inside it, so their number is at most its length.

Parallel workers. Each worker runs one streaming engine and holds back emitted segments until they lie $2w$ behind its stream position, together with the detections of that suffix. This adds at most $2w$ characters plus one segment of at most $w + m + c$ characters. The bound does not depend on $|T|$, and k workers need at most k times as much. □

References

[1] Alneyadi, S., Sithirasenan, E., Muthukkumarasamy, V.: A survey on data leakage prevention systems. Journal of Network and Computer Applications **62**, 137–152 (2016) https://doi.org/10.1016/j.jnca.2016.01.008

[2] Meli, M., McNiece, M.R., Reaves, B.: How bad can it git? Characterizing secret leakage in public GitHub repositories. In: Network and Distributed System Security Symposium (NDSS) (2019). https://doi.org/10.14722/ndss.2019.23418

[3] Krause, A., Klemmer, J.H., Huaman, N., Wermke, D., Acar, Y., Fahl, S.: Pushed by accident: A mixed-methods study on strategies of handling secret information in source code repositories. In: 32nd USENIX Security Symposium, pp. 2527–2544 (2023). https://www.usenix.org/conference/usenixsecurity23/presentation/krause

[4] Microsoft: Presidio: Data Protection and De-identification SDK. https://github

.com/microsoft/presidio. Version 2.2.355 evaluated. Accessed 11 October 2026 (2026)

[5] Gitleaks contributors: Gitleaks. https://github.com/gitleaks/gitleaks. Version 8.28.0 evaluated. Accessed 11 October 2026 (2026)

[6] Truffle Security: TruffleHog. https://github.com/trufflesecurity/trufflehog. Version 3.90.8 evaluated. Accessed 11 October 2026 (2026)

[7] Basak, S.K., Cox, J., Reaves, B., Williams, L.: A comparative study of software secrets reporting by secret detection tools. In: 2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM) (2023). https://doi.org/10.1109/ESEM56168.2023.10304853

[8] Ptacek, T.H., Newsham, T.N.: Insertion, evasion, and denial of service: Eluding network intrusion detection. Technical report, Secure Networks, Inc. (January 1998)

[9] Wang, X., Hong, Y., Chang, H., Park, K., Langdale, G., Hu, J., Zhu, H.: Hyperscan: A fast multi-pattern regex matcher for modern CPUs. In: 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI), pp. 631–648 (2019). https://www.usenix.org/conference/nsdi19/presentation/wang-xiang

[10] Intel Corporation: Introduction to Hyperscan. https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-hyperscan.html. Accessed 11 October 2026 (2019)

[11] Neamatullah, I., Douglass, M.M., Lehman, L.-w.H., Reisner, A., Villarroel, M., Long, W.J., Szolovits, P., Moody, G.B., Mark, R.G., Clifford, G.D.: Automated de-identification of free-text medical records. BMC Medical Informatics and Decision Making **8**, 32 (2008) https://doi.org/10.1186/1472-6947-8-32

[12] Norgeot, B., Muenzen, K., Peterson, T.A., Fan, X., Glicksberg, B.S., Schenk, G., Rutenberg, E., Oskotsky, B., Sirota, M., Yazdany, J., Schmajuk, G., Ludwig, D., Goldstein, T., Butte, A.J.: Protected health information filter (Philter): accurately and securely de-identifying free-text clinical notes. npj Digital Medicine **3**, 57 (2020) https://doi.org/10.1038/s41746-020-0258-y

[13] Stubbs, A., Uzuner, Ö.: Annotating longitudinal clinical narratives for de-identification: The 2014 i2b2/UTHealth corpus. Journal of Biomedical Informatics **58**(Suppl), 20–29 (2015) https://doi.org/10.1016/j.jbi.2015.07.020

[14] Dernoncourt, F., Lee, J.Y., Uzuner, Ö., Szolovits, P.: De-identification of patient notes with recurrent neural networks. Journal of the American Medical Informatics Association **24**(3), 596–606 (2017) https://doi.org/10.1093/jamia/ocw156

[15] Microsoft: presidio-research: Toolbox for developing and evaluating PII detectors. https://github.com/microsoft/presidio-research. Synthetic dataset `synth_dataset_v2.json`, revision 0cb3650. Accessed 11 October 2026 (2026)

[16] Lison, P., Pilán, I., Sánchez, D., Batet, M., Øvrelid, L.: Anonymisation models for text data: State of the art, challenges and future directions. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), pp. 4188–4203 (2021). https://doi.org/10.18653/v1/2021.acl-long.323

[17] Pilán, I., Lison, P., Øvrelid, L., Papadopoulou, A., Sánchez, D., Batet, M.: The text anonymization benchmark (TAB): A dedicated corpus and evaluation framework for text anonymization. Computational Linguistics **48**(4), 1053–1101 (2022) https://doi.org/10.1162/coli_a_00458

[18] Tjong Kim Sang, E.F., De Meulder, F.: Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In: Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003, pp. 142–147 (2003). https://aclanthology.org/W03-0419

[19] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pp. 4171–4186 (2019). https://doi.org/10.18653/v1/N19-1423

[20] Sanh, V., Debut, L., Chaumond, J., Wolf, T.: DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108 (2019). https://doi.org/10.48550/arXiv.1910.01108

[21] Zaratiana, U., Tomeh, N., Holat, P., Charnois, T.: GLiNER: Generalist model for named entity recognition using bidirectional transformer. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pp. 5364–5376 (2024). https://doi.org/10.18653/v1/2024.naacl-long.300

[22] Krawczyk, H., Bellare, M., Canetti, R.: HMAC: Keyed-Hashing for Message Authentication. RFC 2104 (1997). https://doi.org/10.17487/RFC2104

[23] Sweeney, L.: k-anonymity: A model for protecting privacy. International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems **10**(5), 557–570 (2002) https://doi.org/10.1142/S0218488502001648

[24] Sinha, V.S., Saha, D., Dhoolia, P., Padhye, R., Mani, S.: Detecting and mitigating secret-key leaks in source code repositories. In: 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories (MSR), pp. 396–400 (2015). https:

//doi.org/10.1109/MSR.2015.48

[25] Saha, A., Denning, T., Srikumar, V., Kasera, S.K.: Secrets in source code: Reducing false positives using machine learning. In: 2020 International Conference on COMmunication Systems & NETworkS (COMSNETS) (2020). https://doi.org/10.1109/COMSNETS48256.2020.9027350

[26] Feng, R., Yan, Z., Peng, S., Zhang, Y.: Automated detection of password leakage from public GitHub repositories. In: Proceedings of the 44th International Conference on Software Engineering (ICSE) (2022). https://doi.org/10.1145/3510003.3510150

[27] Basak, S.K., Neil, L., Reaves, B., Williams, L.: SecretBench: A dataset of software secrets. In: 2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR) (2023). https://doi.org/10.1109/MSR59073.2023.00053

[28] Shannon, C.E.: A mathematical theory of communication. Bell System Technical Journal **27**(3), 379–423 (1948) https://doi.org/10.1002/j.1538-7305.1948.tb01338.x

[29] Praetorian: Nosey Parker. https://github.com/praetorian-inc/noseyparker. Rule set at revision 2e6e7f3. Accessed 11 October 2026 (2026)

[30] Muthukrishnan, S.: Data streams: Algorithms and applications. Foundations and Trends in Theoretical Computer Science **1**(2), 117–236 (2005) https://doi.org/10.1561/0400000002

[31] Thompson, K.: Regular expression search algorithm. Communications of the ACM **11**(6), 419–422 (1968) https://doi.org/10.1145/363347.363387

[32] Luhn, H.P.: Computer for Verifying Numbers. US Patent 2,950,048 (1960)

[33] Brynjólfsson, S., Jayakrishnan, S., Sali, E., Purwar, D., Aggarwal, M.: RedactionBench. arXiv:2606.18782 (2026). https://doi.org/10.48550/arXiv.2606.18782

[34] Zhu, J., He, S., He, P., Liu, J., Lyu, M.R.: Loghub: A large collection of system log datasets for AI-driven log analytics. In: 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), pp. 355–366 (2023). https://doi.org/10.1109/ISSRE59848.2023.00071

[35] Papadopoulou, A., Lison, P., Øvrelid, L., Pilán, I.: Bootstrapping text anonymization models with distant supervision. In: Proceedings of the Thirteenth Language Resources and Evaluation Conference, pp. 4477–4487. European Language Resources Association, Marseille, France (2022)

[36] Yun, J., Choi, S., Lee, Y., Sokol, O., Shim, W., Melkonyan, A., Kuzmenko, D.:

Project CredData: A Dataset of Credentials for Research. https://github.com/Samsung/CredData. Samsung; repository commit 01e3d13 (2021)

[37] Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap. Chapman & Hall/CRC, New York (1993). https://doi.org/10.1201/9780429246593

[38] Field, C.A., Welsh, A.H.: Bootstrapping clustered data. Journal of the Royal Statistical Society: Series B (Statistical Methodology) **69**(3), 369–390 (2007) https://doi.org/10.1111/j.1467-9868.2007.00593.x